

**Skillab**

# AI Agent Development

**Richard Șerban Francu**

Lead AI Engineer @UCE Systems



# Richard Serban Francu

- Lead AI Engineer @ UCE Systems
- Peste 6 ani de experiență în AI, cu proiecte livrate în producție în FinTech, EdTech și Automotive
- Am construit și pus în producție sisteme reale bazate pe LLM-uri și agenți AI, inclusiv un sistem NL2SQL cu RAG
- Experiență practică cu RAG, fine-tuning, procesare documente și infrastructură AWS (SageMaker, Lambda, Docker)
- Predă din experiență directă de pe piață, nu din teorie academică

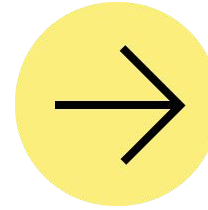
# Să ne cunoaștem! - Menti

Scanează codul QR sau intră pe link-ul de mai jos!

<https://www.menti.com/al73baju8see>

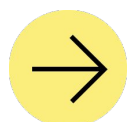


# Programa cursului



1. **Noțiuni de bază — API-uri LLM și primul agent**
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF

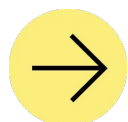
# Teme pentru acasă



Pe durata cursului veți primi **5 teme pentru acasă**

o temă după fiecare 2 lecții, 20p fiecare

Înainte de fiecare temă primești codul și demo-ul de la cele 2 lecții precedent



Participarea la curs **nu este condiționată** de rezolvarea temelor, însă vă încurajez călduros să le completați. Temele valorează **100p** în total:



## **minim 70p**

Vei obține o “Diplomă de Absolvire” care să ateste însușirea competențelor în urma cursului



## **sub 70p**

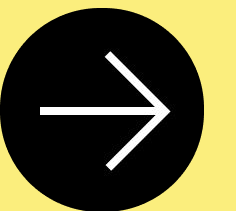
Vei primi un “Certificat de Participare” care să ateste parcurgerea cursului

# Lecția 01

## Notiuni de bază — API-uri LLM și primul agent

- Diferența dintre produsele LLM și API-uri
- Configurarea și utilizarea furnizorilor LLM (Ollama, Gemini, Anthropic)
- Efectuarea apelurilor API folosind Python
- Utilizarea LangChain pentru abstractizarea furnizorului
- Structura unui agent simplu de întrebări și răspunsuri

# LLM Landscape & Fundamente



# Ce este un LLM?

## Definiție

**Rețea neurală** antrenată pe cantități masive de text

Bazat pe arhitectura **Transformer** (Attention)

Prezice **următorul token** pe baza contextului

LLM-uri populare:

| Open Source    | Proprietare        |
|----------------|--------------------|
| Llama (Meta)   | GPT-4 (OpenAI)     |
| Mistral        | Claude (Anthropic) |
| Qwen (Alibaba) | Gemini (Google)    |



***"LLM-urile nu înțeleg —  
prezic continuări  
statistic probabile"***

*Consecință practică:  
nu te baza pe LLM pentru  
fapte precise*

# LLM-urile sunt Estimări Statistice

## Cum învață un LLM:

Minimizează eroarea medie (*loss function*) pe datele de antrenament · Efectiv **comprimă** miliarde de texte · Prezice ce e mai **probabil statistic**, nu ce e adevărat

## Consecințe pentru tine ca dezvoltator:

| Fenomen                 | De ce se întâmplă                                     |
|-------------------------|-------------------------------------------------------|
| Raspunsuri generice     | Modelul tinde spre ce e mai comun/probabil            |
| Fapte specifice greșite | Date, numere, nume rare sunt "blur-ate" în medie      |
| Halucinații             | Generează ce "sună corect" statistic, nu ce e factual |
| Edge cases slabe        | Situațiile rare sunt slab reprezentate în "medie"     |

LLM = bun pentru pattern-uri și raționament · slab pentru lookup exact (date, cifre, nume)

# Produse vs API-uri



## Produse (pentru utilizatori)

**ChatGPT** — OpenAI

**Claude.ai** — Anthropic

**Gemini App** — Google

**Copilot** — Microsoft

*Interfețe gata de utilizare.  
Personalizare limitată.*



## API-uri (pentru dezvoltatori)

**OpenAI API** — OpenAI

**Anthropic API** — Anthropic

**Google AI API** — Google

**Azure OpenAI** — Microsoft

*Control total.  
Construiești aplicații custom.  
Integrare în sisteme.*

**În acest curs: construim cu API-uri, nu cu produse**

# De ce contează API-urile pentru dezvoltatori



**Construiești aplicații custom** — chatboți, asistenți, automatizări



**Integrezi în workflow-uri** — CRM, tickete suport, procesare documente



**Controlezi prompt-urile precis** — system prompts, few-shot examples



**Fără limitări de UI** — procesezi 1000 de documente programatic



**Optimizezi costurile** — alegi modele, cache răspunsuri, batch requests

# Parametri Cheie LLM

|                |                              |                                     |
|----------------|------------------------------|-------------------------------------|
| temperature    | Randomness / creativitate    | 0.0 → deterministic · 2.0 → creativ |
| max_tokens     | Limită lungime răspuns       | 100 → scurt · 4096+ → lung          |
| top_p          | Nucleus sampling             | 0.1 → conservator · 1.0 → deschis   |
| top_k          | Limitează alegerile de token | 1 → greedy · 100 → diversitate      |
| stop_sequences | Când oprești generarea       | ex: ["\n", "END"]                   |

Factual → temperature 0.0–0.3 · Creativ → temperature 0.7–1.0

# LLM-urile NU sunt Deterministe



## Randomness în sampling

temperature > 0 introduce aleatoriu în fiecare token generat

## Variații floating-point

ordinea batch-urilor pe GPU afectează calculele numerice

## Diferențe de infrastructură

servere diferite, hardware diferit → rezultate diferite

Chiar și cu **temperature=0**:

*Majoritatea providerilor nu garantează determinism · Anthropic: "aproape deterministic" · OpenAI: seed există, tot nu e 100%*

**Proiectează mereu pentru variabilitate · Folosește validare output**

# Bias în LLM-uri

## Ce este bias-ul?

Prejudecăți și stereotipuri învățate din datele de antrenament.

LLM-urile reflectă bias-urile din textele de pe internet.

## Ce poți face:

Testează cu diverse inputuri

Prompt-uri care cer obiectivitate

Guardrails pentru output

(→ *Lecția 9*)

## Tipuri de bias:

| Tip        | Exemplu                                         |
|------------|-------------------------------------------------|
| Cultural   | Preferință perspective occidentale/americane    |
| De gen     | "Doctor" → bărbat, "Nurse" → femeie             |
| Temporal   | Cunoștințe mai bogate despre evenimente recente |
| Lingvistic | Performanță mai bună în engleză                 |
| Sycophancy | Tinde să fie de acord cu userul, chiar greșit   |

# Context Length (Lungimea Contextului)

**Context Length** = maximul de tokeni (input + output) pe care modelul îi poate procesa simultan. "**Memoria de lucru**" a modelului.

## Limite de context pe model:

| Model      | Context Length |
|------------|----------------|
| GPT-3.5    | 4K / 16K       |
| GPT-4      | 8K / 128K      |
| Claude 3   | 200K           |
| Gemini 1.5 | 1M / 2M        |
| Llama 3    | 8K / 128K      |

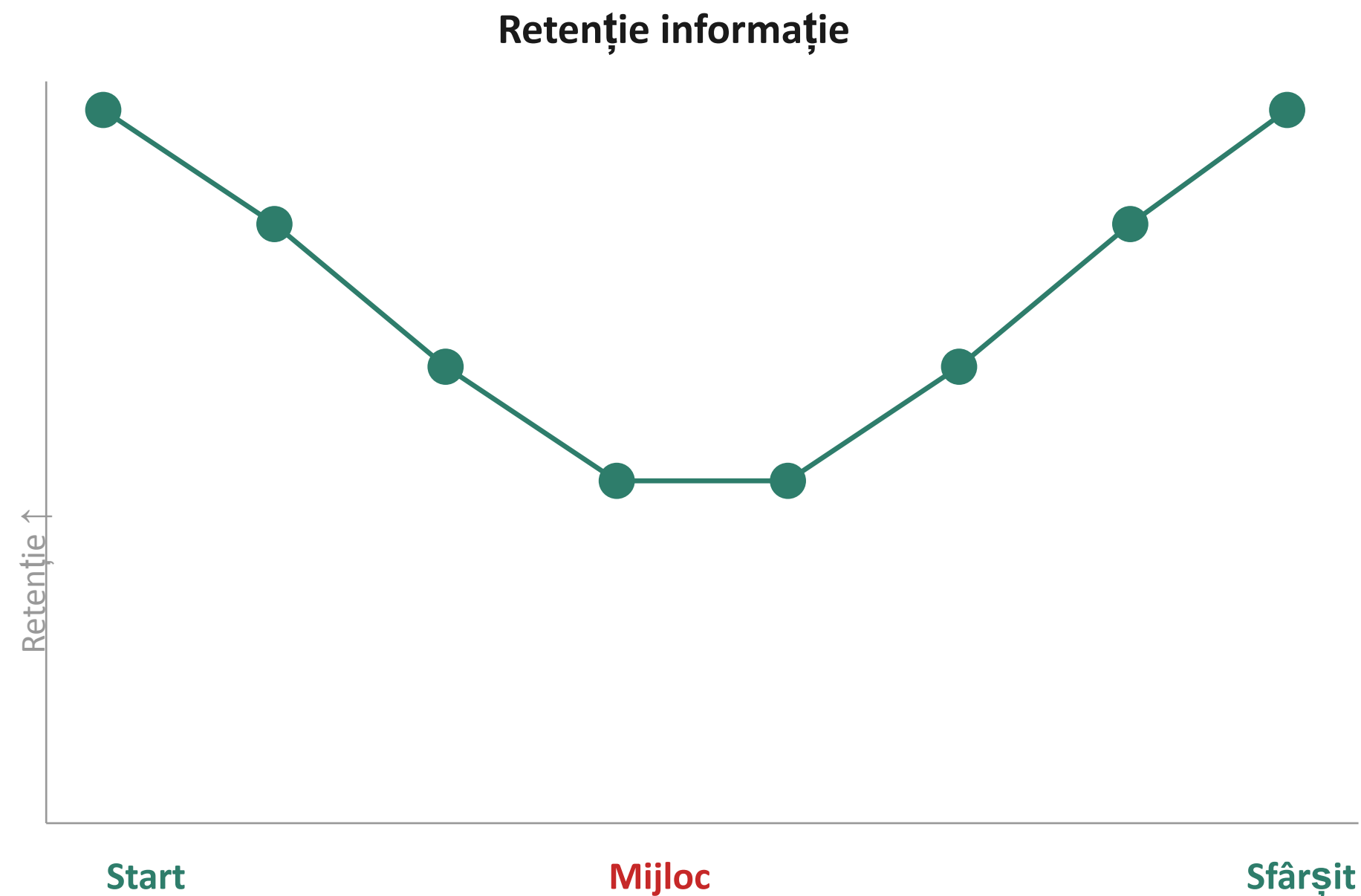
## De ce contează:

-  Determină cât text poți trimite
-  Context mai lung = cost mai mare
-  Context mai lung ≠ rezultate mai bune

→ *vezi slide-ul următor (Lost in the Middle)*

# Lost in the Middle (Pierdut la Mijloc)

Descoperire din cercetare (2023): LLM-urile rețin informația de la ÎNCEPUT și SFÂRȘIT mult mai bine decât din MIJLOC



## Problema:

A pune totul în context  
nu funcționează  
Informația importantă din  
mijloc se "**pierde**"  
Problematic în special cu  
contexte foarte lungi

→ *Soluție: RAG*  
(Lecția 4)

# Best Practices pentru Context

## 1. Structurează prompt-urile:

**ÎNCEPUT** → instrucțiuni importante

**ÎNCEPUT/SFÂRȘIT** → date critice

**MIJLOC** → context mai puțin important

## 2. Pentru documente mari:

- Împarte în bucăți mici (chunking)
- Sumarizează înainte de a include
- Folosește RAG (Lecția 4)

## 3. Design pattern recomandat:

[ System prompt ← ÎNCEPUT ✓ ]

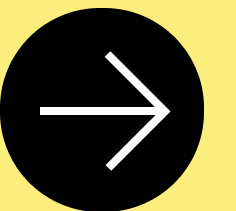
[ Few-shot examples ← ÎNCEPUT ✓ ]

[ Context retrieval ← mijloc ⚠ ]

[ Întrebarea userului ← SFÂRȘIT ✓ ]

[ Reminder format ← SFÂRȘIT ✓ ]

# Provider Overview



# Peisajul Providerilor LLM

## Open Source

Llama (Meta)

Mistral

Qwen (Alibaba)

Gemma (Google)

DeepSeek

## Proprietare

GPT-4 (OpenAI)

Claude (Anthropic)

Gemini (Google)

Grok (xAI)

Command (Cohere)

În acest curs folosim:



modele open source, local



API cloud, free tier



API cloud, premium



# Ollama

## Ce este:

Tool pentru a rula LLM-uri local pe computerul tău  
Suportă: Llama, Mistral, Qwen, Gemma...  
API compatibil OpenAI — ușor de integrat

## Setup:

```
1 bash
2 # 1. Instaleaza Ollama (Linux/macOS)
3 curl -fsSL https://ollama.com/install.sh | sh
4
5 # 2. Descarca modelul si porneste serverul
6 ollama pull llama3.2
7 ollama serve # ruleaza pe localhost:11434
```

## Recomandat pentru:

*Dezvoltare · Învățare · Prototipare*

## Avantaje

- ✓ Complet gratuit
- ✓ Privacy totală (datele nu pleacă)
- ✓ Fără rate limits
- ✓ Funcționează offline

## Dezavantaje

- ✗ Necesită RAM (8GB+ / 16GB+)
- ✗ Mai lent decât cloud (fara GPU)
- ✗ Calitate sub modelele top



# Google Gemini

## Ce este:

API de la Google pentru modelele Gemini  
Free tier generos pentru învățare  
Modele: 2.5 Flash · 2.5 Pro · 2.5 Flash-Lite

## Setup:

```
1 from google import genai
2 # Client conectat automat prin GOOGLE_API_KEY din env
3 client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))
```

## Free Tier Limits (Gemini 2.5 Flash):

| Metric         | Limit   |
|----------------|---------|
| Requests/minut | 10      |
| Tokens/minut   | 250,000 |
| Requests/zi    | 250     |

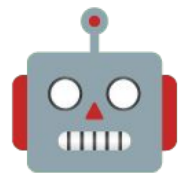
## Avantaje

- ✓ Free tier generos
- ✓ Modele rapide și capabile
- ✓ Context lung (1M+ tokens)
- ✓ Multimodal (text, imagini, video)

## Dezavantaje

- ✗ Date folosite pt. training (free)
- ✗ Rate limiting pe free tier
- ✗ Disponibilitate regională limitată

Recomandat pentru: Învățare · Proiecte mici · Experimente



# Anthropic Claude

## Ce este:

API pentru modelele Claude (Anthropic)  
Focus pe safety și helpfulness  
Modele: Haiku 4.5 · Sonnet 4.5 · Opus 4.5

## Setup:

```
1 import anthropic
2
3 # Client conectat automat prin ANTHROPIC_API_KEY din env
4 client = anthropic.Anthropic()
```

## Pricing (per 1M tokens):

| Model      | Input | Output |
|------------|-------|--------|
| Haiku 4.5  | \$1   | \$5    |
| Sonnet 4.5 | \$3   | \$15   |
| Opus 4.5   | \$5   | \$25   |

## Avantaje

- ✓ Calitate excelentă (top benchmarks)
- ✓ Context 200K tokens (1M pe Opus/Sonnet)
- ✓ Foarte bun la coding și raționament
- ✓ Prompt caching (reduce costuri)

## Dezavantaje

- ✗ Doar platit (no free tier)
- ✗ Rate limits pe tier-ul entry

*Recomandat pentru: Producție · Proiect final · Când calitatea contează*

# Comparație: OpenAI vs Anthropic vs Google

| Aspect         | OpenAI (GPT)       | Anthropic (Claude)      | Google (Gemini)          |
|----------------|--------------------|-------------------------|--------------------------|
| Filosofie      | "Move fast"        | Safety-first, research  | Enterprise, ecosystem    |
| Puncte forte   | Ecosistem, plugins | Rationament, coding     | Multimodal, context lung |
| Context max    | 128K               | 1M                      | 2M                       |
| Coding         | ★★★★               | ★★★★★                   | ★★★★                     |
| Rationament    | ★★★★               | ★★★★★                   | ★★★★                     |
| Prompt caching | Nu                 | Explicit - 90% discount | Implicit/Explicit        |

## Pricing comparativ (per 1M tokens, modele mid-tier):

| Provider  | Model             | Input  | Output |
|-----------|-------------------|--------|--------|
| OpenAI    | GPT-4o            | \$2.50 | \$10   |
| Anthropic | Claude Sonnet 4.5 | \$3    | \$15   |
| Google    | Gemini 2.5 Flash  | \$0.30 | \$2.50 |

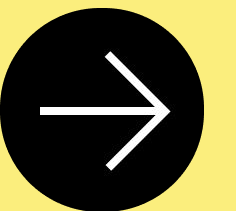
# Comparatie si Recomandari

Recomandare pentru acest curs:

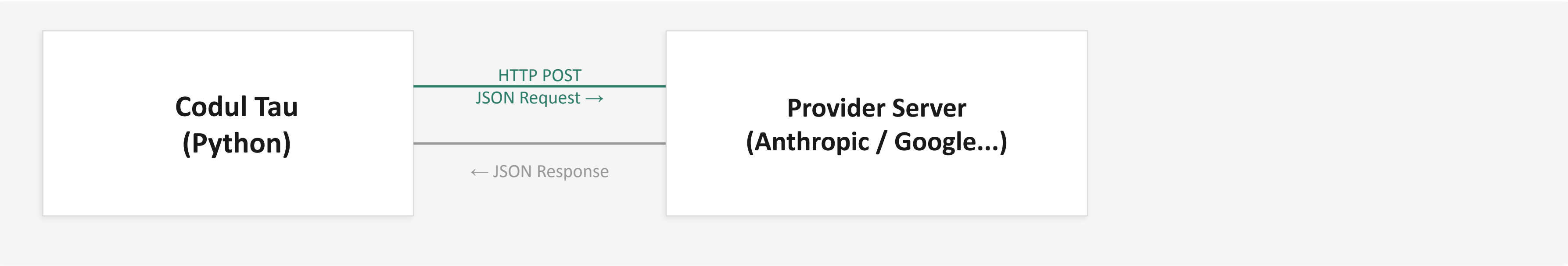
Lecții & Teme → Ollama sau Gemini (gratuit)

Proiect Final → Anthropic Claude (calitate producție)

# API Fundamentals



# Cum Funcționează API-urile LLM



## Caracteristici importante:

| Caracteristică   | Ce înseamnă                                     |
|------------------|-------------------------------------------------|
| Stateless        | Serverul nu ține minte conversații anterioare   |
| Request/Response | Trimiți tot contextul, primești răspuns complet |
| Pay-per-token    | Plaăești pentru input + output tokens           |
| Rate Limited     | Limite pe requests/min, tokens/min              |

Tu ești responsabil sa trimiți ÎNTREGUL istoric al conversației la fiecare request!

# Autentificare și API Keys

## Cum funcționează:

Fiecare provider îți dă un API key (string secret)  
Il trimiți în header-ul HTTP la fiecare request  
Provider-ul te identifică și îți taxează contul

## Unde obții API keys:

| Provider  | URL                                                                                           |
|-----------|-----------------------------------------------------------------------------------------------|
| Anthropic | <a href="https://console.anthropic.com/settings/keys">console.anthropic.com/settings/keys</a> |
| Google    | <a href="https://aistudio.google.com/apikey">aistudio.google.com/apikey</a>                   |
| OpenAI    | <a href="https://platform.openai.com/api-keys">platform.openai.com/api-keys</a>               |

## Reguli de aur:

- ❌ NICIODATĂ în git (.gitignore!)
- ❌ NICIODATĂ în cod sursă
- ❌ NICIODATĂ în logs / print
- ✅ Environment variables / .env

## Best Practices cod:

```
1  # GRESIT - hardcoded in cod
2  api_key = "sk-ant-abc123..."
3
4  # CORECT - environment variable
5  import os
6
7  api_key = os.environ.get("ANTHROPIC_API_KEY")
8
9  # CORECT - fisier .env
10 from dotenv import load_dotenv
11
12 load_dotenv()
13
14 api_key = os.environ.get("ANTHROPIC_API_KEY")
```

# Structură Mesajelor — 3 Roluri

| Rol       | Cine vorbește      | Scop                                             |
|-----------|--------------------|--------------------------------------------------|
| system    | Tu (dezvoltatorul) | Setează comportamentul, personalitatea, regulile |
| user      | Utilizatorul       | Întrebări, comenzi, input                        |
| assistant | LLM-ul             | Răspunsurile modelului                           |



```
1  "messages": [  
2  
3    {"role": "system", "content": "Ești un asistent..."},  
4  
5    {"role": "user", "content": "Ce este o listă?"},  
6  
7    {"role": "assistant", "content": "O listă este..."},  
8  
9    {"role": "user", "content": "Da-mi un exemplu"}  
10 ]  
11  
12
```

## Vizualizare:

**SYSTEM:** "Ești un asistent..." ← *setat de tine*

**USER:** "Ce este o listă?" ← *utilizator*

**ASSISTANT:** "O listă este..." ← *raspuns LLM*

**USER:** "Da-mi un exemplu" ← *utilizator*

# System Prompt

## Ce este:

Instrucțiuni speciale pentru LLM, invizibile pentru utilizator  
Setează personalitatea, regulile, formatul output-ului  
Primul mesaj din conversație (rol system)

## Componente comune:

1. **ROL** "Esti un expert in..."

2. **CONTEXT** "Lucrezi pentru..."

3. **REGULI** "Raspunde doar in..."

4. **FORMAT** "Foloseste markdown..."

5. **RESTRICTII** "Nu discuta despre..."

```
1  """
2  Esti un asistent tehnic
3  pentru dezvoltatori Python.
4
5  CONTEXT:
6  - Lucrezi pentru o companie fintech
7  - Utilizatorii sunt juniori
8
9  REGULI:
10 - Raspunde concis, max 3 paragrafe
11 - Include exemple de cod
12 - Explica termenii tehnici
13
14 RESTRICTII:
15 - Nu oferi sfaturi financiare
16 - Nu executa cod nesigur
17 """
```

# Anatomia Request & Response

## Request (ce trimiți):

```
1 {
2   "model": "claude-sonnet-4-5",
3   "max_tokens": 1024,
4   "temperature": 0.7,
5   "system": "Esti un asistent.",
6   "messages": [
7     {"role": "user", "content": "Salut!"}
8   ]
9 }
```

## Response (ce primești):

```
1 {
2   "id": "msg_abc123",
3   "role": "assistant",
4   "content": [
5     {"type": "text", "text": "Salut! Cu ce te pot ajuta?"}
6   ],
7   "stop_reason": "end_turn",
8   "usage": {
9     "input_tokens": 25,
10    "output_tokens": 15
11  }
12 }
```

## Câmpuri importante în response:

| Câmp        | Ce conține                            |
|-------------|---------------------------------------|
| content     | Textul răspunsului                    |
| stop_reason | end_turn · max_tokens · stop_sequence |
| usage       | Tokeni folositi (pentru calcul cost)  |

# Ce este un Token? Tipuri de Tokenizare

## BPE / Subword

Folosit de: GPT, Claude, Gemini

Împarte în bucăți frecvente, nu neapărat cuvinte întregi.

### Exemple:

"tokenization"

→ [token][ization] = 2

"automatizare"

→ [auto][mat][iz][are] = 4

✓ **Vocabular compact, acopera orice cuvânt**

✗ **Greu de prezis manual câți tokeni**

~1 token = 3/4 cuvânt (en)

## Word Tokenization

Folosit de: NLTK, SpaCy, NLP clasic

Fiecare cuvânt = 1 token. Simplu, intuitiv.

### Exemple:

"Salut lume!"

→ [Salut] [lume] [!] = 3

"running" vs "run"

→ 2 tokeni diferiți!

✓ **Ușor de înțeles și debugat**

✗ **Vocabular urias, cuvinte noi = UNK**

1 token = 1 cuvânt

## Character Tokenization

Folosit de: modele vechi, cod, ADN

Fiecare literă = 1 token. Vocabular minimal.

### Exemple:

"Hello"

→ [H][e][l][l][o] = 5!

"cat" vs "cats"

→ share [c][a][t]

✓ **Vocabular minimal (~256 chars)**

✗ **Secvențe uriase, context epuizat rapid**

1 token = 1 caracter

**LLM-urile moderne folosesc BPE.** Platești per token (input + output).

# Tokens: Română vs Engleză

## De ce româna costă mai mult?

BPE e antrenat pe ~90% text englezesc. Vocabularul e optimizat pentru engleză, nu pentru limbi flecționale.

### Engleză: cuvinte stabile

book / books / book's  
→ 3 forme, rădăcina recunoscută

### Româna: flexiune bogată

carte / cartea / cărții / cărților  
→ 6+ forme, fiecare tăiată diferit

Plus: diacriticele (ă, â, î, ș, ț) sunt rare în corpus → tokenizate ca 2 bytes fiecare.

## Comparație reală EN vs RO

|                           |              |
|---------------------------|--------------|
| "Hello, how are you?"     | 5 tok        |
| "Salut, cum esti?"        | 7 tok (+40%) |
| "based on the agreements" | 5 tok        |
| "in baza intelegerii"     | 8 tok (+60%) |

Regula: RO = 1.5-2x mai mulți tokeni decât EN

## Strategii practice pentru RO

- ★ System prompt în engleză, răspunsuri în română (-30% cost)
- ★ Bugetați 2x tokeni vs tutorialele EN pentru RAG
- ★ Caching devine și mai valoros — discount 90% pe system prompt

Demo live: [tiktokenizer.vercel](https://tiktokenizer.vercel.com) — alege GPT-4 și tokenizează același text în EN și RO

# Prompt Caching

## Problema:

Trimiți același system prompt la fiecare request · Plătești pentru aceeași tokeni iar și iar.

## Soluția: Prompt Caching

**Request 1:** [System: 2000 tokens] + [User: 50 tokens] → **Plătești 2050**

**Request 2:** [System: CACHED] + [User: 60 tokens] → **Plătești 60**

### CACHED PREFIX

(system, examples)

- ✓ Procesat o dată
- ✓ Stocat pe server
- ✓ 90% discount

### DYNAMIC PART

(user message)

- ↻ Procesat mereu  
(dar e mic)

Implementează caching de la început pentru orice system prompt >500 tokens

# Caching pe Provideri

| Aspect     | Anthropic                | Google                   | OpenAI    |
|------------|--------------------------|--------------------------|-----------|
| Nume       | Prompt Caching           | Context Caching          | Automatic |
| Control    | Explicit (cache_control) | Explicit (cachedContent) | Automat   |
| Min tokens | 1,024                    | 32,768                   | N/A       |
| TTL        | 5 minute                 | 1 ora (configurabil)     | Sesiune   |
| Discount   | 90% input                | 75% input                | ~50%      |

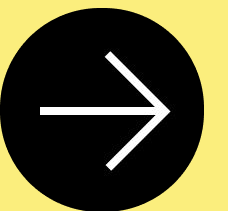
## Anthropic:

```
1 response = client.messages.create(
2     model="claude-sonnet-4-5",
3     max_tokens=1024,
4     system=[
5         {
6             "type": "text",
7             "text": "Continut lung de cache-uit (>1024 tokens)...",
8             "cache_control": {"type": "ephemeral"}, # marcheaza pt. cache
9         }
10    ],
11    messages=[{"role": "user", "content": "Intrebare despre context."}],
12 )
```

## Google:

```
1 # 1. Creezi cache-ul o data
2 cache = client.caches.create(
3     model="gemini-2.5-flash",
4     config={
5         "contents": ["Continut lung de cache-uit (>32K tokens)..."],
6         "ttl": "3600s", # 1 ora
7     },
8 )
9
10 # 2. Il refolosesti la fiecare request (75% reducere input)
11 response = client.models.generate_content(
12     model="gemini-2.5-flash",
13     contents="Intrebare despre context.",
14     config={"cached_content": cache.name},
15 )
```

# Raw API Calls



# De Ce Începem cu Raw API?

LangChain, LlamaIndex, etc.

←  
Abstractizează

HTTP Requests  
(requests, httpx)

← înțelegem asta ACUM

API Endpoints  
(Anthropic, Google, etc.)

```
pip install requests python-dotenv
```

## De ce conteaza:

1

### Debugging

când LangChain dă erori, înțelegi ce se întâmplă

2

### Intelegere

știi exact ce date trimiți și primești

3

### Flexibilitate

poți face lucruri pe care frameworks nu le suportă

4

### Aprecieri

vei înțelege valoarea abstracțiilor

# Raw API: Ollama

## Endpoint:

`localhost:11434/api/chat`

## Particularități:

- ✓ Nu necesită API key
- ✓ Rulează local
- ✓ Format similar OpenAI
- ⚠ Pornit cu: `ollama serve`

## Response text:

```
data["message"]["content"]
```

```
1  import requests
2
3  # Apel raw HTTP catre Ollama local – fara SDK, fara API key
4  response = requests.post(
5      "http://localhost:11434/api/chat",
6      json={
7          "model": "llama3.2",
8          "messages": [
9              {"role": "user", "content": "Salut! Cine esti?"}
10         ],
11         "stream": False,
12     },
13 )
14
15 data = response.json()
16 print(data["message"]["content"]) # textul raspunsului
```

# Raw API: Google Gemini

## Endpoint:

```
generativelanguage
.googleapis.com
/v1beta/models/
{model}:generateContent
```

## Particularități:

- ⚠ API key în URL params
- ⚠ contents → parts (diferit!)
- ⚠ Rol: model, nu assistant
- ✅ system Instruction separat

## Response text:

```
data["candidates"][0]
["content"]["parts"][0]["text"]
```

```
1  import os, requests
2
3  API_KEY = os.getenv("GOOGLE_API_KEY")
4  MODEL = "gemini-2.5-flash"
5
6  # API key in URL params (specific Gemini)
7  url = (
8      f"https://generativelanguage.googleapis.com"
9      f"/v1beta/models/{MODEL}:generateContent?key={API_KEY}"
10 )
11
12 response = requests.post(url, json={
13     # systemInstruction separat de mesaje
14     "systemInstruction": {"parts": [{"text": "Esti un asistent."}]},
15     # contents -> parts (nu "messages -> content" ca la OpenAI)
16     "contents": [
17         {"role": "user", "parts": [{"text": "Salut!"}]},
18         # NB: rolul e "model", nu "assistant"
19     ],
20 })
21
22 data = response.json()
23 print(data["candidates"][0]["content"]["parts"][0]["text"])
```

# Raw API: Anthropic

## Endpoint:

```
api.anthropic.com  
/v1/messages
```

## Particularități:

⚠ Header: anthropic-version

⚠ max\_tokens obligatoriu

✓ system ca și câmp separat

✓ usage info in response

## Response Text:

```
data["candidates"][0]  
["content"]["parts"][0]["text"]
```

```
1  import os, requests  
2  
3  response = requests.post(  
4      "https://api.anthropic.com/v1/messages",  
5      headers={  
6          "x-api-key": os.getenv("ANTHROPIC_API_KEY"),  
7          "anthropic-version": "2023-06-01", # header obligatoriu  
8          "content-type": "application/json",  
9      },  
10     json={  
11         "model": "claude-sonnet-4-5",  
12         "max_tokens": 1024, # obligatoriu la Anthropic  
13         "system": "Esti un asistent.", # camp separat (nu in messages)  
14         "messages": [  
15             {"role": "user", "content": "Salut!"}  
16         ],  
17     },  
18 )  
19  
20 data = response.json()  
21 print(data["content"][0]["text"])  
22 print(data["usage"]) # input_tokens / output_tokens
```

# Diferențele Dureroase — de ce avem nevoie de abstracție

| Aspect        | Ollama                       | Gemini                         | Anthropic                       |
|---------------|------------------------------|--------------------------------|---------------------------------|
| Endpoint      | <code>/api/chat</code>       | <code>:generateContent</code>  | <code>/v1/messages</code>       |
| Auth          | Nimic                        | URL param                      | Header <code>"x-api-key"</code> |
| System prompt | <code>in messages</code>     | <code>systemInstruction</code> | camp <code>"system"</code>      |
| Messages key  | <code>"messages"</code>      | <code>"contents"</code>        | <code>"messages"</code>         |
| Role asistent | <code>"assistant"</code>     | <code>"model"</code>           | <code>"assistant"</code>        |
| Response text | <code>message.content</code> | <code>candidates[0]...</code>  | <code>content[0].text</code>    |

→ Următoarea secțiune: LangChain rezolvă toate aceste diferențe cu o singură interfață

# Problema: Mentenanță Imposibilă

## Fără abstracție:

```
1 # === FARA ABSTRACTIE: cod duplicat pt. fiecare provider ===
2
3 # Ollama
4 import requests
5 r = requests.post("http://localhost:11434/api/chat", json={...})
6 text = r.json()["message"]["content"]
7
8 # Gemini
9 url = f"...{MODEL}:generateContent?key={KEY}"
10 r = requests.post(url, json={"contents": [...]}))
11 text = r.json()["candidates"][0]["content"]["parts"][0]["text"]
12
13 # Anthropic
14 r = requests.post("https://api.anthropic.com/v1/messages",
15     headers={"x-api-key": KEY, "anthropic-version": "2023-06-01"},
16     json={"model": "...", "max_tokens": 1024, "messages": [...]}))
17 text = r.json()["content"][0]["text"]
```

## Probleme:

- Cod duplicat pentru același concept
- Bug fix în 3 locuri
- Testare x3 · Edge cases x3

## Cu LangChain:

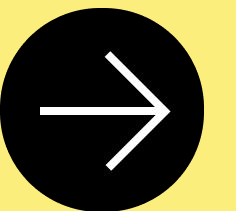
```
1 # === CU LANGCHAIN: acelasi cod, o linie se schimba ===
2 from langchain_ollama import ChatOllama
3 from langchain_google_genai import ChatGoogleGenerativeAI
4 from langchain_anthropic import ChatAnthropic
5
6 # Doar linia asta difera intre provideri
7 llm = ChatOllama(model="llama3.2")
8 # llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
9 # llm = ChatAnthropic(model="claude-sonnet-4-5")
10
11 # Restul codului e IDENTIC pentru orice provider
12 text = llm.invoke("Salut!").content
```

*Același cod  
pentru ORICE provider*

- ✓ O linie se schimbă
- ✓ Zero cod duplicat
- ✓ Un singur loc de fix

→ Următoarea secțiune: LangChain în practică

# Streaming



# De Ce Streaming?

## Fără Streaming

*User: "Explică-mi Python"*

5 secunde de așteptare...

*Utilizatorul nu știe dacă funcționează*

## Cu Streaming

*User: "Explică-mi Python"*

Python

este

un

limbaj

...

*text apare progresiv*

**Feedback instant, experiență naturală**

## Beneficii:

### Perceived speed

pare mai rapid (chiar dacă totalul e același)

### Feedback vizual

utilizatorul vede că "gândește"

### Early stopping

poți opri dacă vezi că merge greșit

### Natural

similar cu felul în care scrie un om

# Cum Funcționează Streaming | SSE

**Client**  
(Python)

HTTP POST →

←----- chunk 1: "Python" -----

←----- chunk 2: " este" -----

←----- chunk 3: " un" -----

←----- [DONE] -----

**Server**  
(Anthropic,  
Google...)

## Format SSE (Server-Sent Events):

```
1 data: {"type": "content_block_delta",
2       "delta": {"text": "Python"}}
3
4 data: {"type": "content_block_delta",
5       "delta": {"text": " este"}}
6
7 data: [DONE]
8
```

## Ce primești la fiecare chunk:

- Un fragment mic de text (1–5 tokeni)
- Metadata (poate include token count)
- La final: semnal de încheiere [DONE]

# Streaming: Cod Exemplu

## SDK: google-genai (async)

```
client.aio.models.generate_content_stream(...)  
# endpoint async
```

```
async for chunk in stream:  
    #delta
```

## Loop de procesare:

- generate\_content\_stream(...) — endpoint async
- async for chunk — iterezi pe măsură ce vin
- if chunk.text — filtrezi chunks goale
- print(chunk.text, flush=True) — afișezi delta

```
1  import asyncio, os  
2  from google import genai  
3  
4  client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))  
5  
6  async def stream_response():  
7      # generate_content_stream — async iterator de chunks  
8      async for chunk in await client.aio.models.generate_content_stream(  
9          model="gemini-2.0-flash",  
10         contents=[{"role": "user", "parts": [{"text": "Scrie un haiku."}]}],  
11         config={"temperature": 0},  
12     ):  
13         if chunk.text:  
14             print(chunk.text, end="", flush=True)  
15  
16  asyncio.run(stream_response())
```

# Streaming: Diferențe între Provideri

| Aspect        | Ollama                       | Gemini                        | Anthropic                   |
|---------------|------------------------------|-------------------------------|-----------------------------|
| Activare      | <code>"stream": true</code>  | <code>stream=True</code>      | <code>"stream": true</code> |
| Format chunk  | JSON lines                   | SSE                           | SSE                         |
| Text location | <code>message.content</code> | <code>candidates[0]...</code> | <code>delta.text</code>     |
| End signal    | <code>"done": true</code>    | <code>finishReason</code>     | <code>message_stop</code>   |

Wrapperul nostru va uniformiza — aceeași interfață pentru toți:

```
1 from abc import ABC, abstractmethod
2 from typing import AsyncIterator
3
4 class LLMPProvider(ABC):
5     """Contract comun pentru orice provider – uniformizeaza diferentele."""
6
7     @abstractmethod
8     async def stream(self, messages: list[Message]) -> AsyncIterator[str]:
9         """Stream de chunks – yield text, indiferent de provider."""
10
11
12 # Folosire – acelasi cod pentru ORICE provider concret
13 async for chunk in provider.stream(messages):
14     print(chunk, end="", flush=True)
```

# Când NU folosești Streaming

| Situație                       | Streaming? | De ce                        |
|--------------------------------|------------|------------------------------|
| Chat interactiv                | Da         | UX mai bun                   |
| Răspunsuri scurte (<50 tokens) | Nu         | Overhead inutil              |
| Structured output (JSON)       | Nu         | Nu poti parsa JSON partial   |
| Batch processing               | Nu         | Vrei să procesezi in bulk    |
| Tool calling                   | Depinde    | Unele frameworks au probleme |
| Logging / Debugging            | Nu         | Mai greu de debugat          |

## Problema cu JSON + streaming:

```
1 # Primești bucati:
2
3 {"name": "Ion
4 ", "age": 25}
5 # json.loads() esueaza!
6
```

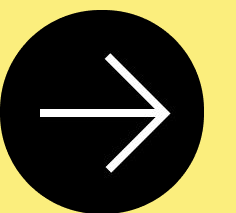
## Recomandare:

**Streaming** — pentru chat UI, output lung

**Non-streaming** — pentru procesare automată

# LangChain & LangGraph

## Introduction



# Problema Abstracției

## Ce am văzut până acum:

### 3 Provideri = 3 Moduri Diferite

- Endpoints diferite
- Autentificare diferită
- Request format diferit
- Response format diferit
- Streaming diferit
- Error handling diferit

## Și asta e doar pentru un simplu chat!

Fara abstracție:

fiecare feature x fiecare provider = cod imposibil de mentinut

## Ce mai vrem sa facem:

### Tools

funcții pe care LLM-ul le poate apela

### Memory

păstrarea istoricului conversației

### Chains

secvențe de operații înlănțuite

### Agents

LLM care decide singur ce să facă

### RAG

retrieval din documente proprii

# Ecosistemul LangChain

## langchain

*Core library*

- Chat Models
- Tools
- Prompts
- Output Parsers
- RAG / Retrievers

## langgraph

*Workflows & State machines*

- State management
- Nodes & Edges
- Conditional routing
- Multi-agent
- Human-in-the-loop

## langsmith

*Observability & Debugging*

- Tracing
- Testing / Evals
- Monitoring
- Dataset management
- Prompt versioning

**langgraph platform** — Deploy & scale agents (inlocuiește LangServe)

# Ce Este LangChain?

**LangChain** = Interfața unificată pentru a lucra cu LLM-uri, indiferent de provider

```
1 llm = ChatAnthropic(model="...")
2 # sau
3 llm = ChatGoogleGenerativeAI(model="...")
4 # Restul codului ramane IDENTIC!
```

| Componenta     | Ce face                    | Exemplu                                |
|----------------|----------------------------|----------------------------------------|
| Chat Models    | Abstracție peste provideri | ChatAnthropic, ChatGoogleGenerativeAI  |
| Messages       | Format unificat mesaje     | HumanMessage, AIMessage, SystemMessage |
| Prompts        | Template-uri reutilizabile | ChatPromptTemplate                     |
| Output Parsers | Parsează răspunsuri        | JsonOutputParser, PydanticOutputParser |
| Tools          | Funcții apelabile de LLM   | @tool decorator                        |
| Retrievers     | Caută în documente         | VectorStoreRetriever                   |

# Ce Este LangGraph?

## Problema cu LangChain simplu:

Chains = secvențe liniare ( $A \rightarrow B \rightarrow C$ )  
Ce faci cu bucle, condiții, ramificări?

**LangGraph** = State Machines pentru Agenți

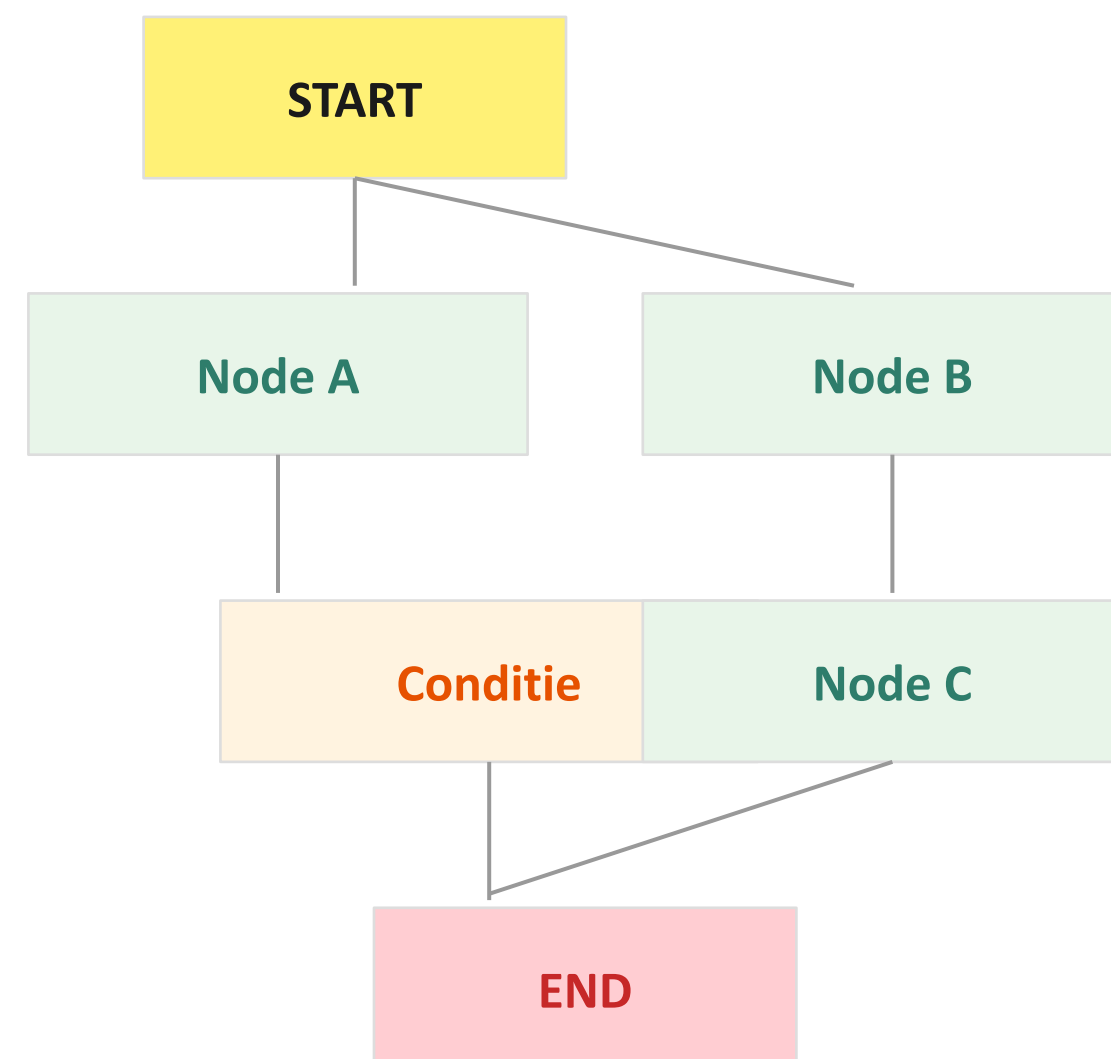
**State** — datele care trec prin graf

**Nodes** — funcții care transformă state

**Edges** — conexiuni între noduri

**Conditional Edges** — ramificări bazate pe condiții

## Graf LangGraph:



# Ce folosești și când?

Simplu chat Q&A?

→ **LangChain e suficient**

Secvență liniară (extract → process → store)?

→ **LangChain chains**

Bucle, retry-uri, condiții complexe?

→ **LangGraph**

Multi-agent cu coordonare?

→ **LangGraph (obligatoriu)**

Agent care decide singur ce tools să folosească?

→ **LangGraph (recomandat)**

In practica: L1-4 → LangChain · L5-6 → LangGraph · Proiect final → ambele

# Alternative la LangChain

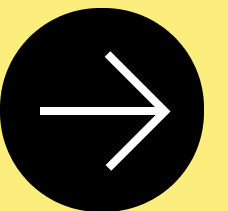
| Framework       | Focus                   | Cand sa-l alegi                                  |
|-----------------|-------------------------|--------------------------------------------------|
| LangChain       | General purpose         | Default — ecosistem mare, documentație excelenta |
| LlamaIndex      | RAG & data              | Cand focusul e pe documente                      |
| Haystack        | Search & RAG            | Enterprise, Elasticsearch                        |
| Semantic Kernel | Microsoft ecosystem     | Azure, .NET, enterprise                          |
| CrewAI          | Multi-agent             | Echipe de agenți colaborativi                    |
| AutoGen         | Multi-agent (Microsoft) | Research, conversații între agenți               |

## De ce alegem LangChain + LangGraph in acest curs:

- ✓ Cel mai mare ecosistem și comunitate
- ✓ Provider-agnostic (funcționează cu oricine)
- ✓ LangGraph cel mai matur pentru state machines
- ✓ LangSmith pentru debugging — util la învățare

Conceptele pe care le înveți se transferă la orice framework!

# LangChain Basics



# Instalare si Setup

## Instalare pachete:

```
1 # Core LangChain
2 pip install langchain langchain-core
3
4 # Provideri
5 pip install langchain-anthropic
6 pip install langchain-google-genai
7 pip install langchain-ollama
8
9 # Utilitati
10 pip install python-dotenv
```

## .env file:

```
1 ANTHROPIC_API_KEY=sk-ant-...
2 GOOGLE_API_KEY=AI...
```

## Încărcare in Python:

```
1 from dotenv import load_dotenv
2 load_dotenv() # incarca .env
3
4 from langchain_anthropic import ChatAnthropic
5 from langchain_google_genai import ChatGoogleGenerativeAI
6 from langchain_ollama import ChatOllama
```

# ChatModel: Primul Apel

Ce returneaza `invoke()`:

## `AIMessage`

tipul obiectului returnat

### `.content`

textul răspunsului (string)

### `.response_metadata`

usage, model, stop reason

Parametri comuni:

```
1
2 # Parametrii comuni la construire
3 llm = ChatGoogleGenerativeAI(
4     model="gemini-2.5-flash",
5     temperature=0.7,
6     max_output_tokens=1024,
7 )
```

```
1 import os
2 from langchain_google_genai import ChatGoogleGenerativeAI
3
4 # Parametrii comuni la construire
5 llm = ChatGoogleGenerativeAI(
6     model="gemini-2.5-flash",
7     google_api_key="key", # cheia din .env
8     temperature=0.7,
9     max_output_tokens=1024,
10 )
11
12 # Apel simplu: string in, AIMessage out
13 raspuns = llm.invoke("Explica ce e un LLM in 2 fraze.")
14
15 print(type(raspuns).__name__) # AIMessage
16 print(raspuns.content)        # textul raspunsului (string)
17 print(raspuns.response_metadata) # usage, model, finish_reas
```

# Messages in LangChain

## Tipuri de mesaje:

### SystemMessage

Instrucțiuni pentru LLM

### HumanMessage

Input de la user

### AIMessage

Răspuns de la LLM

## Construire conversatie:

```
1 from langchain_core.messages import SystemMessage, HumanMessage
2 # Apel simplu: string in, AIMessage out
3 messages = [
4     SystemMessage(content="Esti expert Python."),
5     HumanMessage(content="Ce este o lista?"),
6 ]
7
8 raspuns = llm.invoke(messages)
```

## Continuarea conversatiei:

```
1 messages.append(raspuns) # AIMessage adaugat automat
2 messages.append(HumanMessage(content="Da-mi un exemplu"))
3
4 raspuns = llm.invoke(messages) # LLM vede tot contextul!
5 print(raspuns.content)
```

## Vizualizare mesaje in memorie:

**SystemMessage** "Esti expert Python."

**HumanMessage** "Ce este o lista?"

**AIMessage** "O lista este..."

**HumanMessage** "Da-mi un exemplu"

**AIMessage** "Exemplu: x = [1, 2, 3]"

# Provider Switching

O singură linie se schimbă:

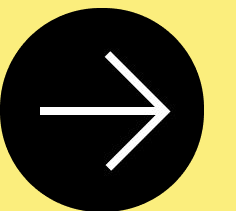
```
1 # Comentezi/decomentezi o singura linie:
2 llm = ChatOllama(model="llama3.2")
3 # llm = ChatAnthropic(model="claude-sonnet-4-5")
4 # llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
5
6 # Restul codului: IDENTIC pentru orice provider
7 print(llm.invoke("Salut!").content)
8
```

Restul codului: IDENTIC

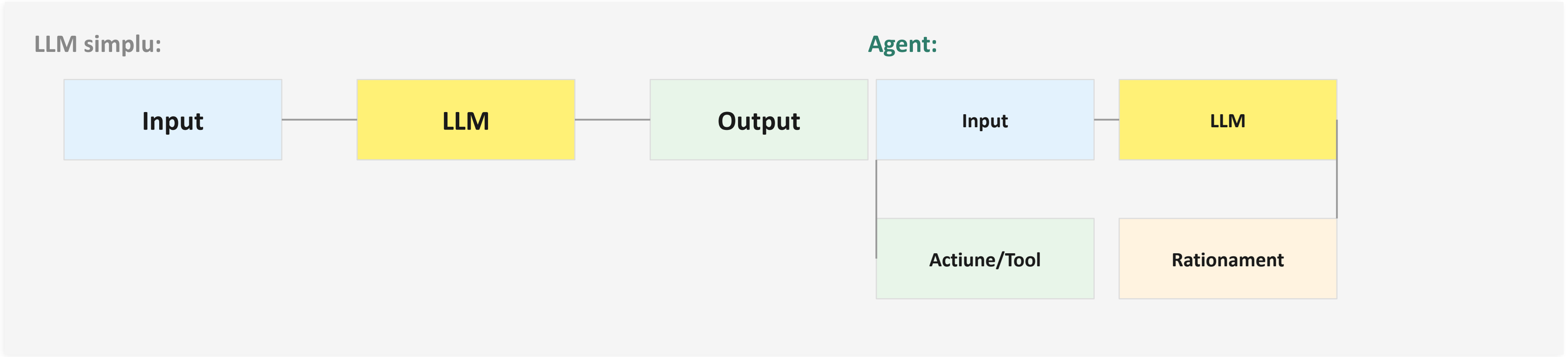
## Factory pattern

```
1 def get_llm(provider: str):
2     """Factory: returneaza un LLM in functie de provider."""
3     if provider == "anthropic":
4         return ChatAnthropic(model="claude-sonnet-4-5")
5     elif provider == "gemini":
6         return ChatGoogleGenerativeAI(model="gemini-2.5-flash")
7     elif provider == "ollama":
8         return ChatOllama(model="llama3.2")
9     raise ValueError(f"Provider necunoscut: {provider}")
10
11 # Schimbi providerul printr-un singur string
12 llm = get_llm("anthropic")
13 print(llm.invoke("Salut!").content)
```

# Building the QA Agent Class



# Ce Este un Agent?



**Agent = LLM + Rationament + Actiuni**

| Componenta    | Ce face                                 |
|---------------|-----------------------------------------|
| LLM           | "Creierul" — generează text, raționează |
| Memory        | Ține minte conversația/contextul        |
| Tools         | Acțiuni pe care le poate executa        |
| Orchestration | Decide ce sa facă next                  |

Astăzi construim: agent Q&A simplu cu memory (fără tools încă)

# De Ce Avem Nevoie de Conversation History?

## Problema — LLM-urile sunt stateless:

```
1 # === PROBLEMA: LLM-urile sunt stateless ===
2 llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
3
4 llm.invoke("Ma cheama Andrei.")
5 raspuns = llm.invoke("Cum ma cheama?")
6 print(raspuns.content)
7 # "Nu stiu cum te cheama..." — apelurile sunt INDEPENDENTE
```

## Solutia — trimitem istoricul:

```
1 history = []
2
3 # Primul tur
4 history.append(HumanMessage("Ma cheama Andrei."))
5 raspuns = llm.invoke(history)
6 history.append(raspuns) # AIMessage adaugat in istoric
7
8 # Al doilea tur — trimitem TOATA conversatia
9 history.append(HumanMessage("Cum ma cheama?"))
10 raspuns = llm.invoke(history)
11 print(raspuns.content)
12 # "Te cheama Andrei." — LLM-ul vede contextul!
```

## Ce trebuie sa gestionam in QAAgent:

### Stocarea mesajelor

lista in memorie (self.history)

### Limita de context

nu poți trimite infinit — vom trata în L8

### Stergerea/resetarea

clear\_history() când e necesar

# QAAgent: Design-ul Clasei

Interfața  
(frontend-ready)

```
1  from langchain_core.language_models.chat_models import BaseChatModel
2  from langchain_core.messages import (
3      BaseMessage, SystemMessage, HumanMessage,
4  )
5
6  class QAAgent:
7      """Agent conversational cu istoric – un wrapper peste un ChatModel."""
8
9      def __init__(self, llm: BaseChatModel, system_prompt: str):
10         self.llm: BaseChatModel = llm
11         self.system_prompt: str = system_prompt
12         self.history: list[BaseMessage] = []
13
14     def chat(self, message: str) -> str:
15         """Trimite mesaj, primește răspuns complet, actualizează istoricul."""
16         ...
17
18     def stream(self, message: str):
19         """Yield chunks pe măsura ce vin de la LLM."""
20         ...
21
22     def get_history(self) -> list[dict]:
23         """Returnează istoricul serializabil."""
24         ...
25
26     def clear_history(self) -> None:
27         """Reset complet al conversației."""
28         self.history = []
```

# QAAgent: Implementare

**\_\_init\_\_:**

**provider**

ollama / anthropic / gemini

**llm**

creat de factory `_create_llm`

**system\_prompt**

comportamentul agentului

**history**

lista goală la început

**\_create\_llm:**

Factory pattern — returnează LLM-ul potrivit. Adaugi un provider nou adaugând un entry in dict.

```
1  class QAAgent:
2      """Agent conversational cu istoric si provider switching."""
3
4      def __init__(self, provider: str, system_prompt: str):
5          self.provider: str = provider
6          self.llm = self._create_llm(provider)          # factory
7          self.system_prompt: str = system_prompt
8          self.history: list[BaseMessage] = []          # lista goala la start
9
10     def _create_llm(self, provider: str):
11         """Factory: returneaza LLM-ul potrivit. Adaugi un provider nou
12         adaugand un entry in dict – fara if/elif crescator."""
13         providers = {
14             "anthropic": lambda: ChatAnthropic(model="claude-sonnet-4-5"),
15             "gemini":    lambda: ChatGoogleGenerativeAI(model="gemini-2.5-flash"),
16             "ollama":    lambda: ChatOllama(model="llama3.2"),
17         }
18         if provider not in providers:
19             raise ValueError(f"Provider necunoscut: {provider}")
20         return providers[provider]() # apelam lambda – instantiere lazy
```

# QAAgent: Metodele Principale

## chat():

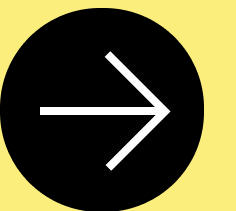
- Construiește [system + history + mesaj]
- Apelează llm.invoke()
- Salvează în history
- Returnează response.content (str!)

## stream():

- Același flow ca chat()
- llm.stream() în loc de invoke()
- yield chunk cu chunk
- Salvează în history DUPA stream complet

```
1  chat(self, message):
2      # 1. Construiește lista completa de mesaje
3      messages = [system_prompt] + history + [user_message]
4
5      # 2. Apel sincron – primești răspunsul întreg
6      response = llm.invoke(messages)
7
8      # 3. Salvează tura în history (user + assistant)
9      history.append(user_message)
10     history.append(response)
11
12     # 4. Returnează textul (string, nu obiectul)
13     return response.content
14
15
16  stream(self, message):
17      # 1. Construiește lista la fel ca la chat()
18      messages = [system_prompt] + history + [user_message]
19
20      # 2. Apel streaming – primești chunks pe rând
21      full_response = ""
22      for chunk in llm.stream(messages):
23          full_response += chunk.content
24          yield chunk.content    # afișare imediată
25
26      # 3. Salvează în history DUPA ce streamul s-a terminat
27      history.append(user_message)
28      history.append(full_response)
```

# Recap & Next Steps



# Ce Am Învățat Astăzi

## - Fundamentele LLM

- Ce sunt LLM-urile si cum funcționează (estimări statistice)
- Parametri cheie: temperature, max\_tokens, etc.
- Limitări: non-determinism, bias, lost in the middle
- Best practices pentru context

## - Hands-on

- Raw API calls la toți 3 providerii
- Streaming responses (SSE)
- LangChain basics și provider switching

★ **QAAgent class** — primul nostru agent!

## Provideri:

Ollama (local) · Gemini (free) · Anthropic (plata)

## De reținut:

LangChain = același cod, orice provider

## - API Fundamentals

- Structura request/response, mesaje: system/user/assistant
- Autentificare și securitate API keys
- Prompt caching pentru optimizare costuri